

AgentDesk User Guide

Complete operating instructions, safety rules, and five practice datasets

Version: Phase 9 development build | Platform: Windows 10/11 x64 | Workspace: C:\projects\agents

What This App Does

AgentDesk is a local Windows desktop app for operating a guarded AI file-writing agent. You type a request into OrchestratorChat. The backend asks OpenAI to produce a short orchestration response, then asks the local worker agent named alice to propose a file operation. File writes do not execute immediately. They appear as a critical approval card so you can inspect, approve, or deny the action.

In the current Phase 9 build, AgentDesk is best used as a controlled practice environment for learning how an AI agent proposes file changes, how approvals work, how snapshots protect the workspace, and how runtime events/errors expose what happened.

Important: AgentDesk is not yet packaged for end-user installation. It runs from the project folder with `npm start`. Packaging/distribution is marked for later Phase 10 work.

Quick Mental Model

Area	Purpose
OrchestratorChat	Enter a natural-language instruction for the agent.
Agent alice	The single local worker agent that proposes file writes.
Approval Card	Shows pending critical action. Approve or deny here.
Typed Confirmation Card	Requires the exact phrase for critical approvals and snapshot restores.
EventStream	Live log of info, warning, and error events.
Snapshot Manager	Create, preview, and restore workspace snapshots.
Cost Counter	Shows token usage reported by OpenAI calls.
Errors	Persistent visible runtime failures.

1. First-Time Setup

Use these steps when starting from the project folder. All commands are PowerShell commands.

1. Open PowerShell.
2. Go to the project folder: `cd C:\projects\agents`
3. Install dependencies if this is a fresh checkout: `npm install`
4. Create or edit the `.env` file in `C:\projects\agents`.
5. Put your OpenAI key in `.env` as `OPENAI_API_KEY=your-key-here`.
6. Optionally set `AGENTDESK_OPENAI_MODEL=gpt-5.4` or another supported model.
7. Start the app with `npm start`.

```
cd C:\projects\agents
npm install
notepad .env
npm start
```

The `.env` file should look like this. Do not include quotes unless your local tooling requires them.

```
OPENAI_API_KEY=your-openai-api-key-here
AGENTDESK_OPENAI_MODEL=gpt-5.4
```

Security rule: never paste real keys into chat, screenshots, issue trackers, or documentation. AgentDesk reads the key in the backend process. The renderer UI only receives `openaiConfigured` `true/false` from `/health`, not the key itself.

How to Know Setup Worked

- The desktop window opens and shows AgentDesk.
- A green toast appears: OpenAI key loaded. AgentDesk is ready for live model calls.
- The header status changes from loading to Backend state loaded.
- The agent list shows alice.
- The EventStream shows database or startup events.

If you do not see the green toast, check that `.env` exists in `C:\projects\agents`, that `OPENAI_API_KEY` has a non-empty value, and that the app was restarted after editing `.env`.

2. Safe Operating Rules

AgentDesk can write files in the alice workspace after you approve a proposed action. Treat the approval step seriously.

- Read the Approval Card before approving.
- Only approve file paths that belong in the workspace.
- Keep Require typed confirmation enabled while learning.
- Create a snapshot before experimenting.
- Use Preview restore before restoring a snapshot.
- Never ask the agent to write `.env` files or secrets. Protected `.env` writes are blocked in this build.

- Do not use the app against important folders until packaging and hardened workspace selection are finished.

Critical confirmation phrase:

I understand this critical action

You must type that phrase exactly before approving a critical write or restoring a snapshot. The phrase is case-sensitive.

3. Understanding the Screen

The screen is split into three columns. The left column is agent and snapshot control, the center column is event history, and the right column is chat, settings, approvals, confirmation, and errors.

Left Column

- Agents: shows alice, her status, model, permissions, and pending queue count.
- Interrupt alice: marks the agent interrupted if you need to stop the current run state.
- Snapshot Manager: captures and restores the workspace folder.
- Cost Counter: tracks input tokens, output tokens, total tokens, and estimated cost.

Center Column

- EventStream: chronological log of backend activity.
- Level filter: show all, info, warn, error, or debug.
- Search events: search event message and source text.
- Raw payload: expand this when debugging exact backend details.

Right Column

- OrchestratorChat: send instructions to the agent.
- Settings: model, max budget, and typed-confirmation setting.
- Approval Card: approve or deny the current pending action.
- Typed Confirmation Card: enter the exact critical phrase.
- Errors: shows runtime errors that need attention.

4. Your First Successful Run

This walkthrough creates a harmless practice file. It is the fastest way to understand the app.

1. Start the app with npm start.
2. Confirm the green OpenAI toast is visible.
3. In Snapshot Manager, name the snapshot Before first practice.
4. Click Create snapshot.
5. In OrchestratorChat, paste the prompt below.

6. Click Send chat.
7. Wait for the Approval Card to show a pending write_file action.
8. Inspect the path. It should be a simple workspace-relative file such as hello-agentdesk.txt.
9. In the Typed Confirmation Card, type: I understand this critical action
10. Click Approve.
11. Check EventStream for Approved write_file executed and Agent run completed.
12. Open C:\projects\agents\workspaces\alice and confirm the file exists.

```
Create a file named hello-agentdesk.txt with this exact content:  
Hello from AgentDesk.  
This is my first approved AI file write.
```

If the Approval Card does not appear, check the Errors panel. Missing or invalid OpenAI keys appear as OPENAI_KEY_MISSING or OPENAI_INVALID_KEY.

5. Daily Workflow

Use this sequence for normal work.

1. Start from a clean app launch.
2. Create a snapshot before trying a new task.
3. Send one clear instruction at a time.
4. Wait for alice to move through running into waiting_for_approval.
5. Review the proposed action in the Approval Card.
6. Approve only if the path and action make sense.
7. Deny if the request is wrong, too broad, unexpected, or unsafe.
8. Use EventStream and Errors to understand failures.
9. Create another snapshot after a good result.

Prompting Tips

- Ask for one file at a time in this build.
- Give an exact file name.
- Give exact content or a clearly bounded structure.
- Avoid vague instructions like improve my project unless you are prepared to inspect the proposed write carefully.
- Use simple relative paths such as notes.md or samples/inventory.json.

Good prompt shape:

```
Create a file named samples/team-notes.md. Include a title, three bullets, and a  
next-actions section.
```

6. Settings

Setting	What it controls	Recommended value
Runtime safety / model	Model name used by the backend OpenAI service.	gpt-5.4 unless you deliberately change it.
Max budget	Runtime budget guard value saved in app settings.	2 for practice; lower if you want stricter limits.
Require typed confirmation	Whether critical actions require the exact confirmation phrase.	Enabled.

To open settings quickly, press Ctrl+, then adjust fields and click Save settings. If you disable typed confirmation, AgentDesk shows a warning banner. Keep it enabled unless you are deliberately testing unrestricted mode.

7. Approvals and Denials

Approvals are the main safety boundary. The agent may propose a write, but the backend waits for your decision.

Choice	Use when	Result
Approve	The file path and content match your intent.	The backend writes the file, creates backups when overwriting, and logs completion.
Deny	The action is wrong, unclear, too broad, or unsafe.	The run is marked denied and alice returns to idle.
Interrupt alice	The agent is stuck or you want to stop the current state.	alice is marked interrupted.

8. Snapshots

Snapshots capture the alice workspace so you can roll back practice changes. They are stored under the workspace in .agentdesk-snapshots and tracked in the local database.

Create a Snapshot

1. Enter a clear snapshot name, such as Before inventory practice.
2. Click Create snapshot.
3. Confirm the snapshot appears in the dropdown.

Preview a Restore

1. Select a snapshot.
2. Click Preview restore.
3. Read the restore diff count. This tells you how many files would change.

Restore a Snapshot

1. Select the snapshot.

2. Type the exact critical confirmation phrase.
3. Click Restore with confirmation.
4. Check EventStream for Snapshot restored.

Snapshot restore validates manifest paths and file hashes before copying content back. If snapshot content is corrupt or tries to escape the workspace, the restore fails instead of silently continuing.

9. EventStream and Errors

The EventStream is your black box recorder. When something feels confusing, read events from oldest to newest and expand Raw payload for details.

Event level	Meaning	Typical action
info	Normal operation completed or progressed.	Use as confirmation.
warn	A guarded, unusual, or operator-facing condition happened.	Inspect before continuing.
error	A runtime operation failed.	Check Errors panel and fix cause.
debug	Low-level diagnostic detail if present.	Use for troubleshooting.

Common Error Codes

Code	Meaning	Fix
OPENAI_KEY_MISSING	No backend API key was available.	Edit .env and restart npm start.
OPENAI_INVALID_KEY	OpenAI rejected the key.	Rotate/fix the key and restart.
CONFIRMATION_MISMATCH	Typed confirmation did not match exactly.	Re-type the exact phrase.
SNAPSHOT_CONFIRMATION_REQUIRED	Snapshot restore needs confirmation.	Type the exact phrase before restore.
PROTECTED_ENV_WRITE	The agent tried to write .env.	Deny the task; do not store secrets through the agent.
WORKER_PLAN_INVALID	The worker did not return valid JSON.	Try a clearer, simpler prompt.

10. Practice Dataset 1: Personal Budget CSV

Goal: learn a clean approve flow and create a small CSV file.

Create a file named samples/budget-may.csv with this exact CSV content:

```
category,planned,actual,notes
Rent,1200,1200,Fixed housing cost
Groceries,350,382,Two extra bulk shops
Transport,120,96,Worked from home twice
Utilities,180,174,Normal month
Savings,500,500,Automatic transfer
```

Practice steps: create a snapshot, send the prompt, approve the write, then open `workspaces\alice\samples\budget-may.csv`. Use EventStream search for virtual-fs to find the write event.

11. Practice Dataset 2: Meeting Notes Markdown

Goal: practice approving a markdown document and checking content formatting.

```
Create a file named samples/project-kickoff-notes.md with these sections:
# Project Kickoff Notes

## Attendees
- Alex
- Priya
- Morgan

## Decisions
- Build a small desktop prototype first.
- Keep file writes behind approvals.
- Store practice outputs in the samples folder.

## Next Actions
- Alex: prepare sample data.
- Priya: test the approval flow.
- Morgan: review the snapshot restore workflow.
```

Practice steps: approve the file, then create a snapshot named After meeting notes. This gives you a restore point after a known-good output.

12. Practice Dataset 3: Inventory JSON

Goal: practice structured data and learn what a worker JSON output can write.

```
Create a file named samples/inventory.json with this exact JSON:
{
  "location": "Workshop A",
  "items": [
    { "sku": "CAB-001", "name": "USB-C cable", "quantity": 18, "reorderAt": 5 },
    { "sku": "DRV-014", "name": "Precision screwdriver", "quantity": 7, "reorderAt": 3 },
    { "sku": "BOX-220", "name": "Storage box", "quantity": 11, "reorderAt": 4 }
  ]
}
```

Practice steps: after approval, open Raw payload on the subagent-worker event. Confirm the proposed path matches `samples/inventory.json`.

13. Practice Dataset 4: Bug Triage Checklist

Goal: practice a workflow file that you can reuse when testing the app.

```
Create a file named samples/bug-triage-checklist.md with this exact content:
# Bug Triage Checklist

- [ ] Reproduce the issue from a fresh app start.
- [ ] Check the Errors panel.
- [ ] Search EventStream by error source.
- [ ] Confirm whether OpenAI is configured.
- [ ] Create a snapshot before retrying risky actions.
- [ ] Record the exact prompt that triggered the issue.
- [ ] Deny any unexpected file write request.
```

Practice steps: deliberately type the wrong confirmation phrase once and observe CONFIRMATION_MISMATCH. Then type the exact phrase and approve.

14. Practice Dataset 5: Restore Drill

Goal: learn snapshot preview and restore without touching important files.

```
Create a file named samples/restore-drill.txt with this exact content:
Restore drill version 1
This file exists so I can practice snapshots safely.
```

1. Create a snapshot named Restore drill v1.
2. Send another prompt asking the agent to overwrite samples/restore-drill.txt with version 2 text.
3. Approve the overwrite only if the path is exactly samples/restore-drill.txt.
4. Preview restore for Restore drill v1.
5. Type the exact critical phrase and restore.
6. Open the file and confirm it is back to version 1.

```
Overwrite samples/restore-drill.txt with this exact content:
Restore drill version 2
This changed version should disappear after I restore the snapshot.
```

15. Troubleshooting

Problem	Likely cause	What to do
App says OpenAI not detected	.env missing, empty, or app not restarted.	Edit .env, save it, close app, run npm start again.
Approval button fails	Typed confirmation mismatch.	Use exact phrase: I understand this critical action.
No approval appears	OpenAI request failed or worker returned invalid JSON.	Check Errors and EventStream; simplify prompt.
Snapshot restore fails	Missing confirmation, invalid snapshot, or hash/path validation failure.	Preview first; use exact phrase; choose a valid snapshot.

Problem	Likely cause	What to do
npm start fails	Port conflict, dependency issue, or backend startup failure.	Close old AgentDesk processes; check npm-start logs.
File not where expected	Workspace-relative path may differ.	Check C:\projects\agents\workspaces\alice.

16. Current Limits and Best Use

- This is a Phase 9 development build, not an installer-ready product.
- There is one built-in worker agent: alice.
- The worker currently supports proposed `write_file` actions.
- The backend binds to 127.0.0.1:9403.
- The renderer cannot access Node APIs and does not receive API keys.
- Packaged backend startup is not finished until Phase 10.
- Use it for controlled local file-writing practice, approval flow training, snapshot drills, and UI validation.

Best practice: keep all practice prompts aimed at the samples folder until you are comfortable reading approvals, events, and restore previews.

17. Full-Potential Learning Path

1. Run Dataset 1 to learn the basic approve flow.
2. Run Dataset 2 to learn markdown outputs and post-success snapshots.
3. Run Dataset 3 to inspect structured JSON output and raw event payloads.
4. Run Dataset 4 to learn confirmation failure and recovery.
5. Run Dataset 5 to learn safe overwrite and restore.
6. After each run, search EventStream by source: orchestrator, subagent-worker, virtual-fs, snapshot-manager, and openai-service.
7. After every risky experiment, create or restore a snapshot so you understand the workspace safety loop.

Appendix: Command Reference

```
cd C:\projects\agents
npm install
npm start
npm test
npm run phase9:check
```

Health check while app is running:

```
Invoke-WebRequest -UseBasicParsing http://127.0.0.1:9403/health
```

Expected safe health shape:

```
{"ok":true,"service":"agentdesk-backend","phase":9,"openaiConfigured":true}
```

Notice that the health response reports only whether a key is configured. It must never print the key.